

UNIT39: IMAGE-BASED LIGHTING

【学習要項】

- IBL : Image-based lighting
- Pre-filtered environment map

【演習手順】

1. シェーダーの実装 (変更)

① HLSL ヘッダーファイルの追加 (image_based_lighting.hlsl)

```
1: #ifndef __IMAGE_BASED_LIGHTING__
2: #define __IMAGE_BASED_LIGHTING__
3:
4: Texture2D skybox : register(t32);
5: TextureCube diffuse_iem : register(t33);
6: TextureCube specular_pmrem : register(t34);
7: Texture2D lut_ggx : register(t35);
8:
9: float4 sample_lut_ggx(float2 brdf_sample_point)
10: {
11:     return lut_ggx.Sample(sampler_states[LINEAR], brdf_sample_point);
12: }
13: float4 sample_skybox(float3 v, float roughness)
14: {
15:     const float PI = 3.14159265358979;
16:     uint width, height, number_of_levels;
17:     skybox.GetDimensions(0, width, height, number_of_levels);
18:
19:     float lod = roughness * float(number_of_levels - 1);
20:
21:     v = normalize(v);
22:     // Blinn/Newell Latitude Mapping
23:     float2 sample_point;
24:     sample_point.x = (atan2(v.z, v.x) + PI) / (PI * 2.0);
25:     sample_point.y = 1.0 - ((asin(v.y) + PI * 0.5) / PI);
26:     return skybox.SampleLevel(sampler_states[LINEAR], sample_point, lod);
27: }
28: float4 sample_diffuse_iem(float3 v)
29: {
30:     return diffuse_iem.Sample(sampler_states[LINEAR], v);
31: }
32: float4 sample_specular_pmrem(float3 v, float roughness)
33: {
34:     uint width, height, number_of_levels;
35:     specular_pmrem.GetDimensions(0, width, height, number_of_levels);
36:
37:     float lod = roughness * float(number_of_levels - 1);
38:
39:     return specular_pmrem.SampleLevel(sampler_states[LINEAR], v, lod);
40: }
41:
42: #endif
```

② bidirectional_reflectance_distribution_function.hlsl で image_based_lighting.hlsl をインクルードする

③ bidirectional_reflectance_distribution_function.hlsl に関数を追加する

```
1: float3 ibl_radiance_lambertian(float3 N, float3 V, float roughness, float3 diffuse_color, float3 f0)
2: {
3:     float NoV = clamp(dot(N, V), 0.0, 1.0);
4:
5:     float2 brdf_sample_point = clamp(float2(NoV, roughness), 0.0, 1.0);
6:     float2 f_ab = sample_lut_ggx(brdf_sample_point).rg;
7:
8:     float3 irradiance = sample_diffuse_iem(N).rgb;
9:
10:     float3 fr = max(1.0 - roughness, f0) - f0;
```

UNIT39: IMAGE-BASED LIGHTING

```
11: float3 k_s = f0 + fr * pow(1.0 - NoV, 5.0);
12: float3 fss_ess = k_s * f_ab.x + f_ab.y;
13:
14: float ems = (1.0 - (f_ab.x + f_ab.y));
15: float3 f_avg = (f0 + (1.0 - f0) / 21.0);
16: float3 fms_ems = ems * fss_ess * f_avg / (1.0 - f_avg * ems);
17: float3 k_d = diffuse_color * (1.0 - fss_ess + fms_ems);
18:
19: return (fms_ems + k_d) * irradiance;
20: }
21: float3 ibl_radiance_ggx(float3 N, float3 V, float roughness, float3 f0)
22: {
23:     float NoV = clamp(dot(N, V), 0.0, 1.0);
24:
25:     float2 brdf_sample_point = clamp(float2(NoV, roughness), 0.0, 1.0);
26:     float2 f_ab = sample_lut_ggx(brdf_sample_point).rg;
27:
28:     float3 R = normalize(reflect(-V, N));
29:     float3 specular_light = sample_specular_pmrem(R, roughness).rgb;
30:
31:     float3 fr = max(1.0 - roughness, f0) - f0;
32:     float3 k_s = f0 + fr * pow(1.0 - NoV, 5.0);
33:     float3 fss_ess = k_s * f_ab.x + f_ab.y;
34:
35:     return specular_light * fss_ess;
36: }
```

④ピクセルシェーダー(gltf_model_ps.hlsl)の main 関数を変更する

※UNIT.38 1. ③ 88: に下記コードを追加する

```
1: diffuse += ibl_radiance_lambertian(N, V, roughness_factor, c_diff, f0);
2: specular += ibl_radiance_ggx(N, V, roughness_factor, f0);
```

2. テクスチャのロード

①framework クラスにメンバ変数を定義する

```
Microsoft::WRL::ComPtr<ID3D11ShaderResourceView> shader_resource_views[8];
```

②framework クラスの initialize メンバ関数でテクスチャをロードする

※学内サーバーから「environments.zip」をコピー・展開する

```
1: D3D11_TEXTURE2D_DESC texture2d_desc;
2: load_texture_from_file(device.Get(), L"..\\environments\\sunset_jhbcentral_4k\\sunset_jhbcentral_4k.dds",
3:     shader_resource_views[0].GetAddressOf(), &texture2d_desc);
4: load_texture_from_file(device.Get(), L"..\\environments\\sunset_jhbcentral_4k\\diffuse_iem.dds",
5:     shader_resource_views[1].GetAddressOf(), &texture2d_desc);
6: load_texture_from_file(device.Get(), L"..\\environments\\sunset_jhbcentral_4k\\specular_pmrem.dds",
7:     shader_resource_views[2].GetAddressOf(), &texture2d_desc);
8: load_texture_from_file(device.Get(), L"..\\environments\\lut_ggx.dds",
9:     shader_resource_views[3].GetAddressOf(), &texture2d_desc);
```

③ framework クラスの render メンバ関数でテクスチャをバインドする

```
1: immediate_context->PSSetShaderResources(32, 1, shader_resource_views[0].GetAddressOf());
2: immediate_context->PSSetShaderResources(33, 1, shader_resource_views[1].GetAddressOf());
3: immediate_context->PSSetShaderResources(34, 1, shader_resource_views[2].GetAddressOf());
4: immediate_context->PSSetShaderResources(35, 1, shader_resource_views[3].GetAddressOf());
```

3. ビルド・実装し、描画されることを確認する

※モデルデータを「DamagedHelmet」以外に変更してテストする

※テクスチャをミップマップ化することでクオリティーが向上する（参照 UNIT.31）

【評価項目】

□IBL の効果